密　级：公开

合肥工業大學

Hefei University of Technology

OpenHarmony 标准版适配启智 01 开发者套件（基于昇腾 310B）并完成 NPU 推理功能技术文档

TECHNICAL DOCUMENT

类　　型：	技术文档
队　　长：	潘孝成
成　员 1：	杨洋
成　员 2：	芦甲翔
指导教师：	李琳　张本宏
学院名称：	软件学院
专业名称：	软件工程
完成时间：	2026 年 8 月

文档更新说明

版本	版本更新说明	撰写人	完成日期
V1.0	初始版本，撰写第一章移植相关的内容	杨洋	2026 年 05 月 18 日
V1.1	补充源码编译流程、已知问题修复方法及镜像制作与烧录步骤	杨洋	2026 年 05 月 25 日
V1.2	新增昇腾 CANN 推理引擎移植章节，安装部署、环境变量配置及 NPU 验证	杨洋	2026 年 06 月 03 日
V1.3	新增 YOLOv5 目标检测推理应用开发章节，ONNX 转 OM、CLI 部署与推理流程	潘孝成	2026 年 06 月 10 日
V1.4	完善 YOLOv5 CLI 代码结构说明、输出张量兼容性及批量推理功能	潘孝成	2026 年 06 月 18 日
V1.5	新增第四章内存优化章节	潘孝成	2026 年 06 月 21 日
V1.6	新增第四章时间优化章节	芦甲翔	2026 年 06 月 22 日
V1.7	完善第四章节内存优化和时间优化的相关内容	潘孝成	2026 年 06 月 27 日
V1.8	全文校对与格式统一	杨洋	2026 年 06 月 28 日
V1.9	增加文档图片展示，全文校对与格式统一，补充参考链接，完成最终版本	潘孝成	2026 年 06 月 30 日
V1.10	新增附录：HDMI 音频排查与构建修复记录（DT compatible 大小写、ninja 内核跳过、工具链 PATH 修复、HDC USB Gadget 启动修复）；第三章补充 YOLOv5 App 实时摄像头检测与原生 NPU 推理说明	杨洋	2026 年 08 月 12 日
V1.11	补充单应用专用系统二次裁剪、HDMI 音频完整链路、DRM/EDID 显示修复、Camera/ImageReceiver 生命周期修复、CANN 运行时随镜像部署与 5001 推理服务自启动，以及 GN/Ninja 构建闭环验证	杨洋	2026 年 08 月 14 日
V1.12	新增 OHOS 4000/Ninja 构建故障排查记录，补充重复输出、条件编译、BPF action 返回值及 sanitizer 配置修复，并记录 GN 与目标构建验证命令	杨洋	2026 年 08 月 17 日

目 录

一 移植记录 1
1.1 搭建开发环境 1
1.1.1 硬件环境 1
1.1.2 安装库和工具集 1
1.1.3 配置 git 账户信息 2
1.1.4 安装码云 repo 工具 2
1.1.5 安装内核交叉编译工具链 3
1.2 源码下载 3
1.2.1 从版本分支获取源码 3
1.2.2 从版本发布的 Tag 节点获取源码 4
1.2.3 预编译文件下载 4
1.3 额外厂家源码下载与替换 4
1.4 编译制作镜像 4
1.4.1 权限添加 4
1.4.2 编译 5
1.4.3 动态分支的编译问题修复 5
1.4.4 Tag 快照编译的问题修复 5
1.4.5 镜像制作 6
1.5 编译和烧录 6
1.6 系统启动与桌面展示 7
二 昇腾 CANN 推理引擎移植 8
2.1 推送文件 8
2.1.1 推送脚本文件 8
2.1.2 推送 CANN 安装包 8
2.2 安装 CANN 组件 9
2.3 安装 Python 推理依赖 9
2.4 配置 CANN 环境变量 10
2.5 验证 NPU 和 pyACL 可用 11
2.6 脚本输出结果 11

三 YOLOv5 目标检测推理应用开发 12
3.1 总体流程 12
3.2 准备 YOLOv5 ONNX 模型 12
3.3 ONNX 转 OM 14
3.3.1 配置虚拟环境 14
3.3.2 安装 CANN-toolkit 15
3.3.3 配置环境变量 16
3.3.4 模型转换 17
3.4 部署 YOLOv5 CLI 到 OpenHarmony 设备 18
3.5 安装设备端 Python 依赖 19
3.6 运行 YOLOv5 目标检测 19
3.7 批量图片推理 20
3.8 PC 端冒烟测试 21
3.9 YOLOv5 CLI 代码结构说明 21
3.10 输出张量兼容性 22
3.10.1 标准 YOLOv5 输出 22
3.10.2 包含 NMS 的输出 23
3.11 进阶：OpenHarmony 图形应用 23
3.11.1 实时摄像头检测功能说明 24
3.11.2 CANN 运行时随镜像部署与推理服务自启动 25
四 系统性能优化与指标测试 27
4.1 概述 27
4.2 内存优化 27
4.2.1 优化思路 27
4.2.2 补丁说明 27
4.2.3 优化效果 29
4.3 启动时间优化 30
4.3.1 优化思路 30
4.3.2 补丁说明 30
4.3.3 优化效果 31

4.4 验证测试 · · · · · 32
4.4.1 内存优化验证 · · · · · 32
4.4.2 启动时间验证（bootchart） · · · · · 32
4.5 优化总结 · · · · · 34
附录 1 参考链接 · · · · · 36
1.1 官方文档与社区 · · · · · 36
1.2 硬件平台与昇腾文档 · · · · · 36
1.3 AI 模型与工具 · · · · · 36
1.4 系统调优参考 · · · · · 37
1.5 资源下载 · · · · · 37
1.6 演示视频 · · · · · 37
附录 2 系统源码二次排查与集成修复记录 · · · · · 38
2.1 背景与目标 · · · · · 38
2.2 关键结论 · · · · · 38
2.2.1 drv_hdmi 的形态与来源 · · · · · 38
2.2.2 根因一：设备树 compatible 大小写不匹配 · · · · · 39
2.2.3 根因二：修改 dtsi 后内核动作被 ninja 跳过 · · · · · 39
2.2.4 根因三：内核交叉编译工具链不在 PATH · · · · · 39
2.3 验证证据 · · · · · 40
2.3.1 产物时间戳 · · · · · 40
2.3.2 新版 dtb 反编译结果 · · · · · 40
2.3.3 关键排查命令 · · · · · 41
2.4 已落地的修改 · · · · · 41
2.5 遇到的问题与排查过程 · · · · · 41
2.5.1 镜像中没有 drv_hdmi.ko 的疑问 · · · · · 41
2.5.2 为什么改完源码再编译还是无效 · · · · · 42
2.5.3 内核 defconfig 编译失败 · · · · · 42
2.6 HDC USB Gadget 启动修复记录 · · · · · 42
2.6.1 问题发现 · · · · · 42
2.6.2 排查思路 · · · · · 42
2.6.3 解决过程 · · · · · 43

2.6.4 验收标准 44
2.7 开机黑屏（显示管线提交失败）排查记录 45
2.7.1 实机结论 45
2.7.2 根因 45
2.7.3 V1.11 源码修复 45
2.7.4 显示验收 46
2.8 V1.11 HDMI 音频全链路补充 46
2.9 Camera 连续分析与 ImageReceiver 生命周期修复 47
2.10 CANN/NAPI 系统镜像集成补充 48
2.11 构建配置闭环与完成状态 48
2.12 OHOS 4000 Ninja 构建故障排查记录 49
2.12.1 问题现象与正确的定位入口 49
2.12.2 排查思路 49
2.12.3 根因与修复 50
2.12.4 验证结果 51
2.12.5 当前构建命令 51

一　移植记录

1.1 搭建开发环境

310b_rc_24.1.rc2_5.0.3 支持由源码编译，进行源码编译前，请根据如下步骤完成相关环境准备。

1.1.1 硬件环境

准备一台 X86 架构、预装 Ubuntu 系统的设备，硬盘容量不小于 200GB。

1.1.2 安装库和工具集

可参考官方相关链接[3]；这里以 Ubuntu 22.04 为例：

(1) 安装基础依赖

```
#安装基础依赖
sudo apt update
sudo apt upgrade
sudo apt install -y \
  openjdk-11-jdk \
  ccache \
  libelf-dev \
  dwarves \
  genext2fs \
  u-boot-tools \
  mtools \
  mtd-utils \
  scons \
  gcc-arm-none-eabi \
  build-essential \
  ninja-build \
  ruby \
  git \
  curl \
  unzip \
  python3 \
  python3-pip \
  bc \
  flex \
  bison \
  libssl-dev
```

代码清单 1.1　安装基础依赖

(2) 安装编译工具

```
#安装编译工具
sudo apt-get update
sudo apt-get install build-essential
sudo apt-get install libncurses*
sudo apt-get install lzop
sudo apt-get install net-tools
sudo apt-get install openssh-server vim git fakeroot
```

代码清单 1.2　安装编译工具

(3) 安装扩展依赖

```
#安装扩展依赖
sudo apt-get install git ssh make gcc libssl-dev liblz4-tool
   expect \
  g++ patchelf chrpath gawk texinfo chrpath diffstat binfmt-
     support \
  qemu-user-static live-build bison flex fakeroot cmake gcc-
     multilib \
  g++-multilib unzip device-tree-compiler python3-pip
     libncurses5-dev
sudo apt-get install libgmp-dev libmpc-dev libicu-dev
   bsdmainutils
sudo apt-get -y apt-utils binutils bison flex bc build-
   essential make \
  mtd-utils gcc-arm-linux-gnueabi u-boot-tools python3-pip git
      zip \
  unzip curl wg
```

代码清单 1.3　安装扩展依赖

1.1.3 配置 git 账户信息

```
#Git配置
git config --global user.name "HFUT_QDZDD"
git config --global user.email "example@mail.hfut.edu.cn"
git config --global credential.helper store
ssh-keygen -t ed25519 -C "Gitcode SSH Key"
```

代码清单 1.4　Git 配置

1.1.4 安装码云 repo 工具

```
#安装Repo
mkdir ~/bin
curl https://raw.gitcode.com/gitcode-dev/repo/raw/main/repo-
   py3 -o ~/bin/repo
chmod a+x ~/bin/repo
pip3 install -i https://repo.huaweicloud.com/repository/pypi/
```

```
    simple requests
#将repo添加到环境变量
vim ~/.bashrc
export PATH=~/bin:$PATH
source ~/.bashrc
```

代码清单 1.5　安装 Repo

1.1.5 安装内核交叉编译工具链

复制 Ascend-hdk-310b-sdk-soc_24.1.rc2.zip Ascend HDK 24.1.RC2 中的 toolchain.tar.gz 至 /opt，并解压。

```
#配置工具链
sudo mkdir /opt/compiler
sudo sh -c 'echo "export PATH=/opt/compiler/toolchain/bin:\
    $PATH" >> /etc/profile'
source /etc/profile
cp toolchain/ /opt/compiler/ -rf
```

代码清单 1.6　配置工具链

1.2 源码下载

执行如下命令，下载 310b_rc_24.1.rc2_5.0.3 仓源码：通过 repo + ssh 下载（需注册公钥，请参考码云帮助中心[4]）。

1.2.1 从版本分支获取源码

可获取该版本分支的最新源码，包括版本发布后在该分支的合入（后文称该分支为动态分支）。

```
#下载分支源码
export REPO_URL='https://mirrors.tuna.tsinghua.edu.cn/git/git-
    repo'
repo init -u git@gitee.com:openharmony/manifest.git \
  -b OpenHarmony-5.0.3-Release --no-repo-verify
repo sync -c
repo forall -c 'git lfs pull'
```

代码清单 1.7　下载分支源码

PS. 网络问题解决方案：清理已下载但不完整的仓库，重新单线程同步。

1.2.2 从版本发布的 Tag 节点获取源码

可获取与版本发布时完全一致的源码（后文称该分支为发布分支，该分支问题较多，不建议使用）。

```
#下载Tag源码
repo init -u git@gitee.com:openharmony/manifest.git \
  -b refs/tags/OpenHarmony-v5.0.3-Release --no-repo-verify
repo sync -c
repo forall -c 'git lfs pull'
```

代码清单 1.8　下载 Tag 源码

1.2.3 预编译文件下载

```
#先进入源码目录，然后下载预编译文件
cd ./build
sudo ./prebuilts_download.sh
```

代码清单 1.9　下载预编译文件

1.3 额外厂家源码下载与替换

- 下载 package/Ascend-hdk-310b-npu-driver-soc_24.1.rc2_linux-aarch64.run 文件[18]。
- 下载 package/Ascend310B-source-bjjc-ohos.tar.gz 文件（脚本可能执行失败，要根据报错手动解压）[19]。
- 下载 device_board_ascend/ascend/310B/ascendinstall/libaarch64.tar.gz 文件[20]。

1.4 编译制作镜像

1.4.1 权限添加

执行以下命令添加执行权限：

```
chmod +x ./device/board/ascend/310B/kernel/build_kernel.sh
chmod +x ./device/board/ascend/310B/kernel/make-ohos.sh
```

代码清单 1.10　添加执行权限

1.4.2 编译

厂家已经提供一键式编译安装能力，进入本源码仓代码根目录，执行 build.sh，后面跟 OpenHarmony 源码根目录，这里以/opt/ohos 举例：

```
#一键编译
sudo ./build.sh /opt/ohos
```

代码清单 1.11　一键编译

编译完成后会在 /opt/ohos/out/310B/packages/phone/images 目录下生成Ascend-hdk-310b-soc_24.1.rc2-harmony_5.0.3-aarch64.zip 包。

PS. 编译失败补丁：可能需要打入下面的 patch：

```
https://gitee.com/phytium_embedded/phytium-openharmony-device/
   blob/5.x_release/device_board_phytium/common/patch/0032=
   foundation-arkui-napi=fix_libjsvm_may_compile_fail.patch
```

代码清单 1.12　编译失败补丁

为了减少每次编译时都要设置环境变量，可以在 build.sh 中添加环境配置：

```
echo "export PATH=/opt/compiler/toolchain/bin:\$PATH: " >> /
   etc/profile
source /etc/profile
```

代码清单 1.13　环境配置

1.4.3 动态分支的编译问题修复

- 0042=foundation-multimedia-image_effect=fix-egl-function-by-eglGetProcAddress 补丁失败，请手动添加（行数不对应）。0042 补丁已提交为 task1_transplantation/patch_fix/foundation-multimedia-image_effect-frameworks-native-render_environment-graphic-gl_utils.cpp.patch。

1.4.4 Tag 快照编译的问题修复

- 0042=foundation-multimedia-image_effect=fix-egl-function-by-eglGetProcAddress 补丁失败，请手动添加（行数不

对应)。0042 补丁已提交为 `task1_transplantation/patch_fix/foundation-multimedia-image_effect-frameworks-native-render_environment-graphic-gl_utils.cpp.patch`。

- **jsvm 语法错误：** `5.0.3/foundation/arkui/napi/jsvm/build_jsvm_inter.sh: 157`。修复：在 `"true"` 后面打一个空格。
- **参数错误：** `base/hiviewdfx/hiview/plugins/eventlogger/log_catcher/peer_binder_catcher.cpp:408`。修复：`DoExecHiperf(fileName, pids, processId, perfCmd)` 删除后面两个参数。
- **参数错误：** `/foundation/window/window_manager/window_scene/session_manager/src/scene_session_manager.cpp:2396` 。修复：去除一个参数。

1.4.5 镜像制作

下载制卡工程 device_board_ascend/310BFormatTest 到 Ubuntu 环境，根据 README 指示, 复制相关库到/usr/lib64/目录下。将编译生成镜像 Ascend-hdk-310b-soc_24.1.rc2-harmony_5.0.3-aarch64.zip 解压到 310BFormatTest/firmware。将 SD 卡连上虚拟机或者服务器，执行如下命令:

```
#复制相关库，添加权限
cp tools/library/* /usr/lib64/
chmod 777 tools/emmc-head

#安装arm64交叉编译 C 标准库
sudo apt install -y libc6-dev-arm64-cross

#制卡命令
cd 310BFormatTest
./formatSD310B.sh /dev/sdX
```

代码清单 1.14　制卡命令

1.5 编译和烧录

以上步骤完成后，之后的编译可参考提交的 `scripts/build_and_flash_310B.sh` 脚本进行一键编译和烧录。

如果修改了内核的相关配置，编译前请删除 out 目录，再进行编译，否则可能修改不生效。

1.6 系统启动与桌面展示

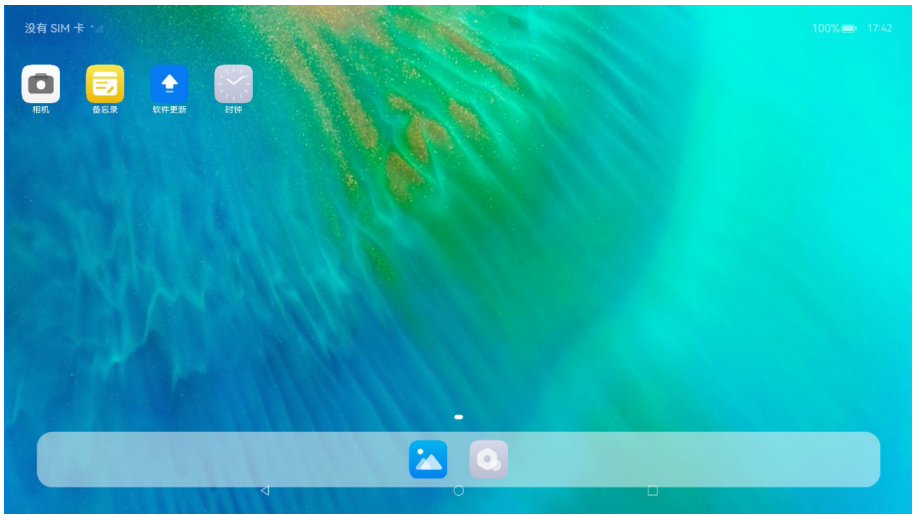


图 1.1　桌面展示



图 1.2　分辨率展示

二　昇腾 CANN 推理引擎移植

本章介绍把华为昇腾 CANN [11]移植到 OpenHarmony 系统。

2.1 推送文件

2.1.1 推送脚本文件

在 Windows PowerShell 中进入当前仓库的 task2_cann 目录，执行部署脚本，将任务 2 相关的安装和验证脚本推送到设备端。

```
#执行脚本部署
.\00_deploy_scripts_from_windows.ps1
```

代码清单 2.1　执行脚本部署

该脚本会把以下文件推送到设备:

```
/cann-install/task2_cann/02_install_cann_on_device.sh
/cann-install/task2_cann/03_set_cann_env.sh
/cann-install/task2_cann/04_install_python_wheels.sh
/cann-install/task2_cann/05_verify_npu.sh
/cann-install/task2_cann/verify_cann_runtime.py
```

代码清单 2.2　推送的脚本文件

2.1.2 推送 CANN 安装包

在 Windows PowerShell 中执行包推送命令，将 CANN 相关安装包从 PC 端传输到设备端。`D:\install` 为 CANN 包在 PC 端的位置。

```
#执行包推送
.\01_push_packages_from_windows.ps1 -PackageDir D:\install
```

代码清单 2.3　执行包推送

此外，为了节省装环境的步骤，本组对 initenv.tar.gz 压缩包进行了修改，在 python_packs 文件夹下面增加了两个.whl 包，分别是 `ais_bench-0.0.2-py3-none-any.whl` 和 `aclruntime-0.0.2-cp310-cp310-linux_aarch64.whl` 这两个包，原在 `example.tar.gz` 中。

脚本会将包推送至设备端以下位置:

```
/storage/temp/initenv.tar.gz
/storage/temp/toolkit.tar.gz
/storage/temp/nnal.tar.gz
/storage/temp/kernels.tar.gz
```

代码清单 2.4　设备端包路径

2.2 安装 CANN 组件

通过 hdc 连接设备，执行 CANN 安装脚本，完成 CANN 推理引擎的完整安装部署。

```
#执行安装脚本
sh /cann-install/task2_cann/02_install_cann_on_device.sh
```

代码清单 2.5　安装 CANN

脚本内部完成以下操作:

- 重挂载根文件系统为可写
- 创建 /cann-install 和 /usr/local/Ascend
- 解压 initenv.tar.gz
- 执行 initmodel.sh 初始化环境
- 安装 /usr/local/Python
- 解压 toolkit.tar.gz
- 逐个安装 toolkit 子包
- 安装 NNAL/ATB
- 安装 kernels/opp_kernel
- 清理临时安装包

PS. 安装问题：如果 initmodel.sh 报 /var/dmp_daemon 相关错误，打开脚本，屏蔽第一行或对应调用后重新执行。

2.3 安装 Python 推理依赖

`/cann-install/python_packs` 已经由 initenv.tar.gz 压缩包解压得到，现在只用执行 Python 依赖安装脚本。

```
#执行脚本，进行Python依赖安装
sh /cann-install/task2_cann/04_install_python_wheels.sh
```

代码清单 2.6　安装 Python 依赖

该脚本会安装指导文档中列出的依赖：

```
pip3 install attrs-24.2.0-py3-none-any.whl
pip3 install PyYAML-6.0.2-cp310-cp310-manylinux_2_17_aarch64.whl
pip3 install wheel-0.44.0-py3-none-any.whl
pip3 install psutil-6.1.0-cp36-abi3-manylinux_2_17_aarch64.whl
pip3 install numpy-1.24.0-cp310-cp310-manylinux_2_17_aarch64.whl
pip3 install decorator-5.1.1-py3-none-any.whl
pip3 install mpmath-1.3.0-py3-none-any.whl
pip3 install sympy-1.13.3-py3-none-any.whl
pip3 install scipy-1.14
……
pip3 install numpy==1.22.4
pip3 install opencv-python==4.10.0.84
pip3 install ipywidgets
pip3 install scikit-video
pip3 install ais_bench-0.0.2-py3-none-any.whl
pip3 install aclruntime-0.0.2-cp310-cp310-linux_aarch64.whl
```

代码清单 2.7　依赖包列表

PS. 兜底安装策略：如果个别 wheel 包版本或文件名不同，脚本最后会遍历安装 `WHEEL_DIR/*.whl`，仍可覆盖大部分离线包场景。

2.4 配置 CANN 环境变量

每次新开设备 Shell 或设备重启后，运行推理前都要执行环境变量配置脚本。

```
#设备Shell配置环境变量
. /cann-install/task2_cann/03_set_cann_env.sh
```

代码清单 2.8　配置环境变量

该脚本配置以下环境变量：

```
ASCEND_TOOLKIT_HOME=/usr/local/Ascend/8.2.RC1
ASCEND_HOME_PATH
ASCEND_AICPU_PATH
ASCEND_OPP_PATH
TOOLCHAIN_HOME
PATH
PYTHONPATH
LD_LIBRARY_PATH
```

代码清单 2.9　环境变量

2.5 验证 NPU 和 pyACL 可用

执行 NPU 和 CANN 运行时验证脚本，确认推理环境正常可用。

```
#设备Shell执行验证脚本
sh /cann-install/task2_cann/05_verify_npu.sh
```

代码清单 2.10　验证 NPU

该脚本会先尝试执行 NPU 信息查询：

```
npu-smi info
```

代码清单 2.11　NPU 信息查询

然后运行 Python ACL 验证：

```
python3.10 /cann-install/task2_cann/verify_cann_runtime.py --
    device-id 0
```

代码清单 2.12　Python ACL 验证

PS. 验证成功输出：`device_count=1`、`run_mode=...`、`CANN runtime verification passed.`，说明 CANN Python ACL 运行时已能初始化 NPU。

2.6 脚本输出结果

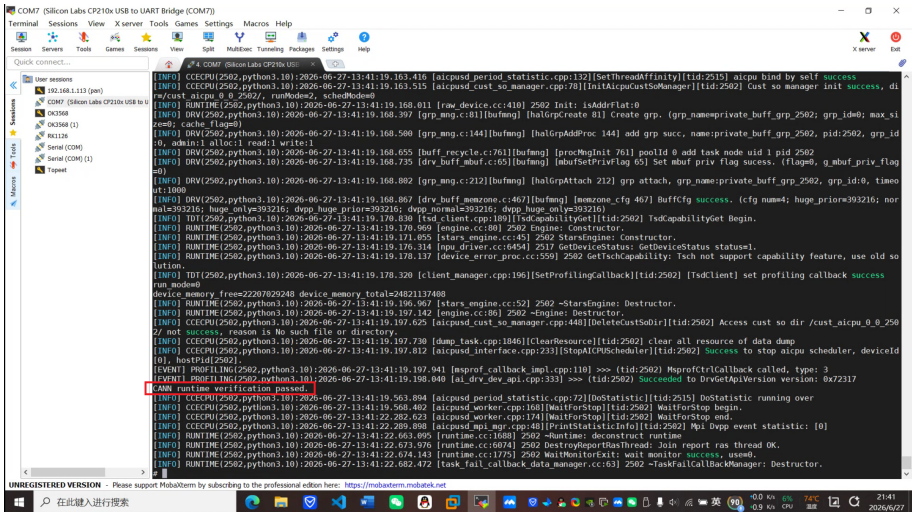


图 2.1　脚本验证结果

三　YOLOv5 目标检测推理应用开发

3.1 总体流程

任务 3 完整链路如下：

```
准备 YOLOv5 代码
  -> 生成权重文件
  -> 导出 onnx 模型
  -> 使用 ATC 转换 OM
  -> 通过 HDC 部署 OM、图片和 Python 程序
  -> 设备端 source CANN 环境变量
  -> Python CLI 读取图片
  -> letterbox 预处理
  -> ais_bench 调用 CANN OM 推理
  -> YOLOv5 NMS 后处理
  -> 绘制检测框
  -> 保存结果图片、txt 和 json
  -> hdc file recv 拉回结果
```

代码清单 3.1　总体流程

CLI 程序入口为：

```
/cann-install/yolov5_cli/main.py
```

代码清单 3.2　CLI 入口

该程序支持两种后端：

- `ais_bench`：设备端 OM/NPU 推理。
- `onnxruntime`：PC 端 CPU 冒烟测试，可用于验证预处理和后处理逻辑。

3.2 准备 YOLOv5 ONNX 模型

由于本组机器性能不够，所以模型生成和转换部分在云服务器上进行，使用的云服务器提供商为恒源云。首先在云服务器上克隆 YOLOv5 源码[14]，然后进行训练，生成.pt 权重文件，接着导出为 ONNX 模型。

```
#clone 源码
git clone https://github.com/ultralytics/yolov5.git
cd yolov5

#环境依赖安装
pip install -r requirements.txt

#训练生成权重文件
```

```
python train.py --data coco128.yaml --weights '' --cfg yolov5s
    .yaml --img 640

#导出 onnx 模型文件
python export.py --weights yolov5s.pt --include onnx --opset
    12 --imgsz 640 --dynamic --simplify
```

代码清单 3.3　导出 ONNX

运行完成后即可获得模型文件：`yolov5s.onnx`

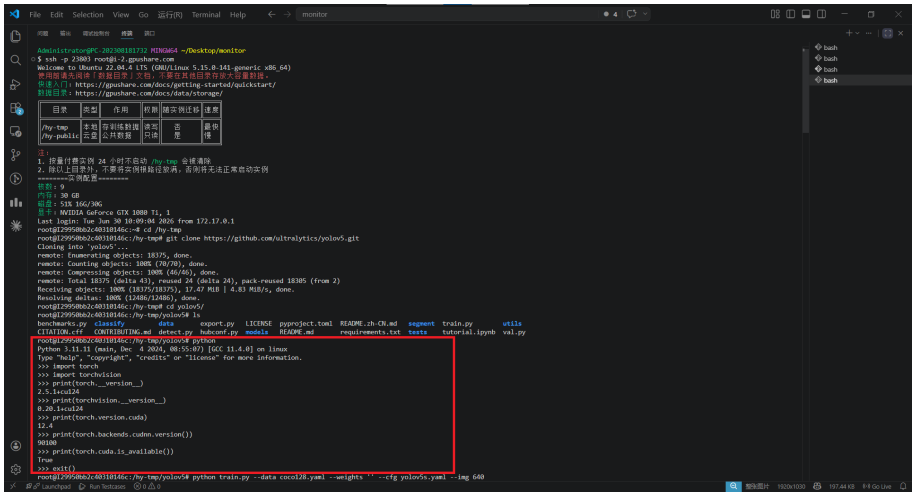


图 3.1　验证 PyTorch 的可用性

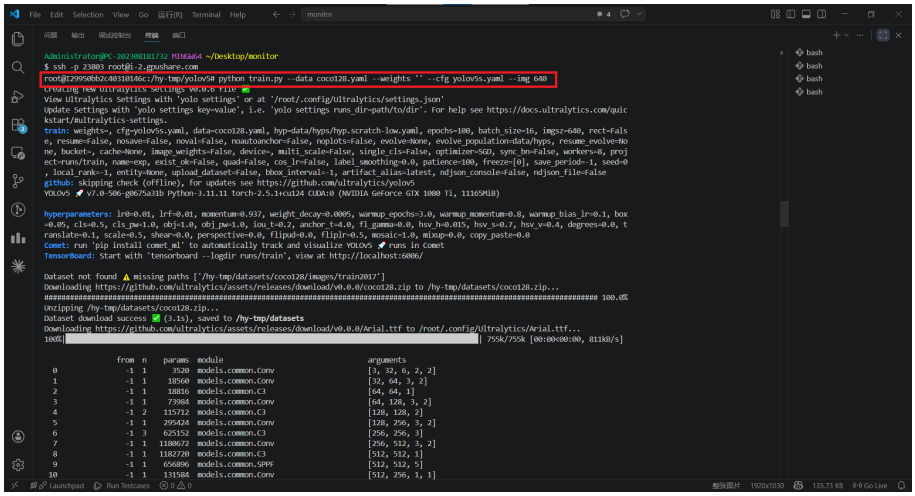


图 3.2　开始训练

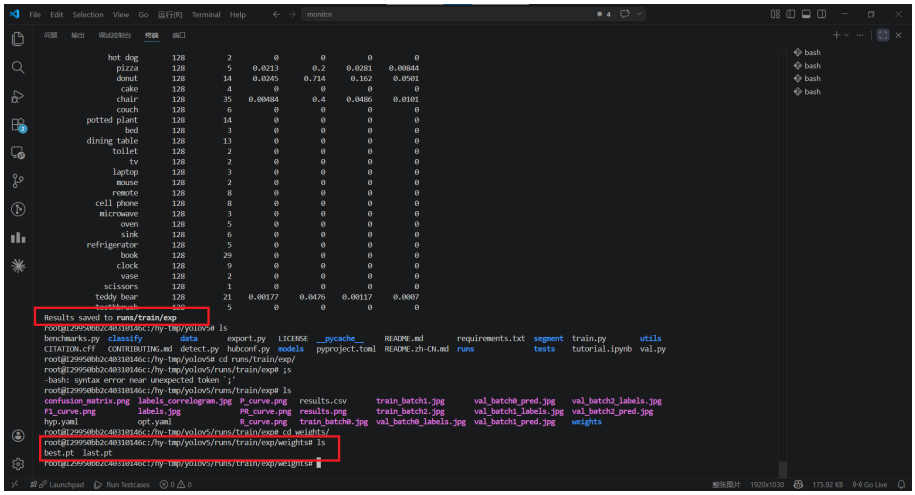


图 3.3　pt 文件获得

3.3 ONNX 转 OM

3.3.1 配置虚拟环境

首先在云服务器中安装 Anaconda 或者 Miniconda，二选一，本组选择装 Anaconda。

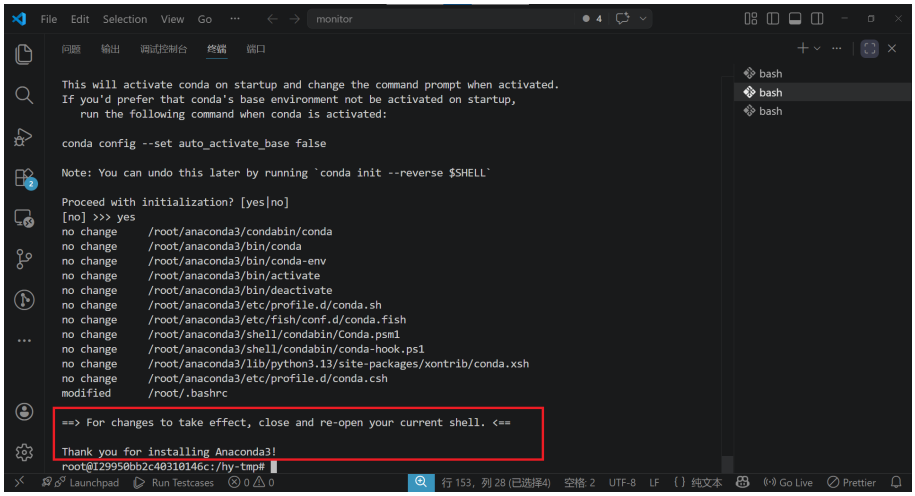


图 3.4　Anaconda 安装结果

安装完成后，配置虚拟环境：

```
#创建环境
conda create -n onnx-to-om python=3.7.5

#激活环境
conda activate onnx-to-om

#进入环境后，安装python依赖
sudo apt-get install -y gcc g++ make cmake zlib1g zlib1g-dev \
  openssl libsqlite3-dev libssl-dev libffi-dev libbz2-dev \
  libxslt1-dev unzip pciutils net-tools libblas-dev gfortran libblas3
```

```
#安装python库
pip install attrs -i https://pypi.tuna.tsinghua.edu.cn/simple
pip install numpy -i https://pypi.tuna.tsinghua.edu.cn/simple
pip install decorator -i https://pypi.tuna.tsinghua.edu.cn/simple
pip install sympy -i https://pypi.tuna.tsinghua.edu.cn/simple
pip install cffi -i https://pypi.tuna.tsinghua.edu.cn/simple
pip install pyyaml -i https://pypi.tuna.tsinghua.edu.cn/simple
pip install pathlib2 -i https://pypi.tuna.tsinghua.edu.cn/simple
pip install psutil -i https://pypi.tuna.tsinghua.edu.cn/simple
pip install protobuf -i https://pypi.tuna.tsinghua.edu.cn/simple
pip install scipy -i https://pypi.tuna.tsinghua.edu.cn/simple
pip install requests -i https://pypi.tuna.tsinghua.edu.cn/simple
pip install absl-py -i https://pypi.tuna.tsinghua.edu.cn/simple
```

代码清单 3.4　创建虚拟环境

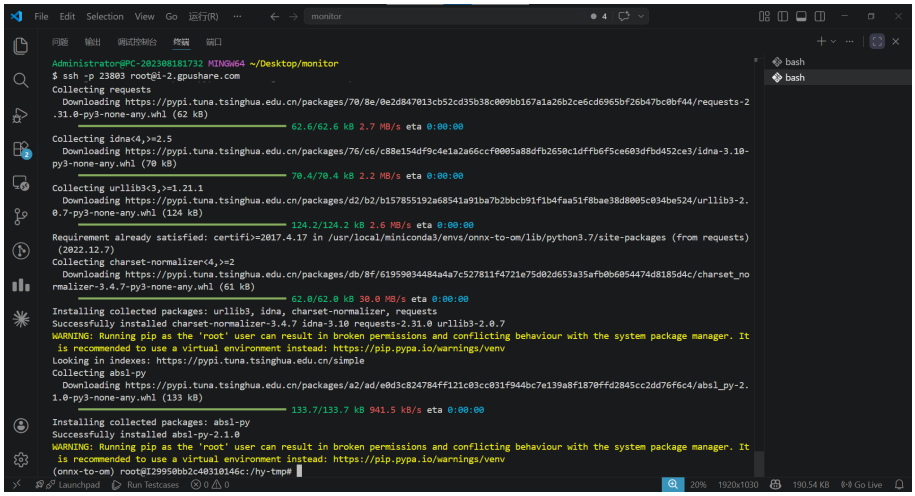


图 3.5　虚拟环境安装与配置

3.3.2 安装 CANN-toolkit

接下来下载所需 CANN 文件，由于在云服务器上只是转换文件，所以只需要 CANN_toolkit 一个软件包，下载 8.2.RC1 版本的 CANN[15]。

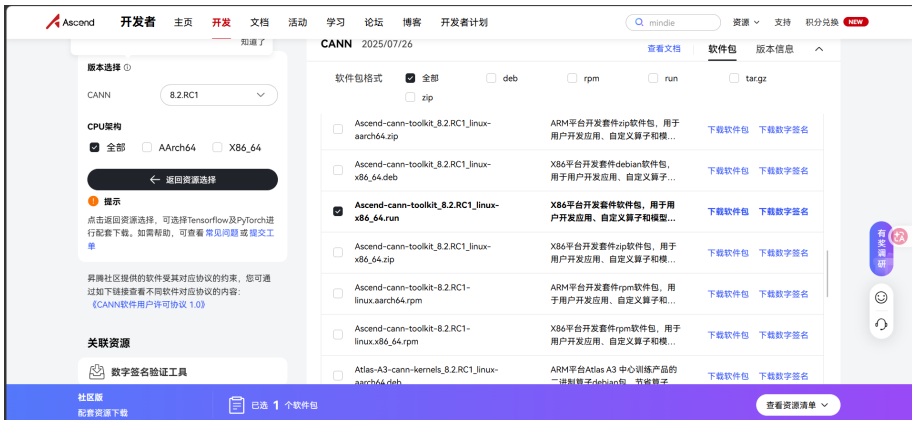


图 3.6　CANN 下载页面

下载完成后，创建个 CANN 文件夹，用于存放文件。

```
#进入文件夹
cd CANN

# 添加执行权限
sudo chmod +x Ascend-cann-toolkit_8.2.RC1.alpha003_linux-
   x86_64.run

# 检查一下文件
./Ascend-cann-toolkit_8.2.RC1.alpha003_linux-x86_64.run --
   check

# 安装文件
./Ascend-cann-toolkit_8.2.RC1.alpha003_linux-x86_64.run --
   install
```

代码清单 3.5　安装 CANN-toolkit

3.3.3 配置环境变量

在脚本末尾添加下面语句, CANN_INSTALL_PATH 是 CANN 软件安装目录。

```
#编辑bashrc
gedit ~/.bashrc

# 例如
source CANN_INSTALL_PATH/ascend-toolkit/set_env.sh
export LD_LIBRARY_PATH=CANN_INSTALL_PATH/ascend-toolkit/latest
   /x86_64-linux/devlib/:$LD_LIBRARY_PATH

# 实际
source /home/om/Ascend/ascend-toolkit/set_env.sh
export LD_LIBRARY_PATH=/home/om/Ascend/ascend-toolkit/latest/
   x86_64-linux/devlib/:$LD_LIBRARY_PATH

# 更新一下脚本
source ~/.bashrc
```

代码清单 3.6　配置 CANN 环境变量

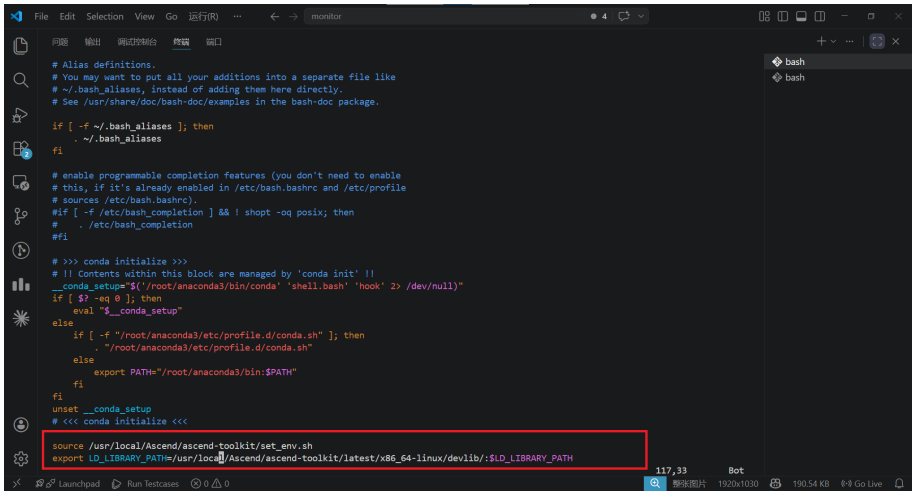


图 3.7　环境变量设置

设置完环境变量后，即可在终端输入 atc 命令，验证安装是否成功，此处 ATC run failed 是因为没有进入相应的虚拟环境，缺少环境依赖导致的，进入 onnx-to-om 虚拟环境后即可恢复正常。

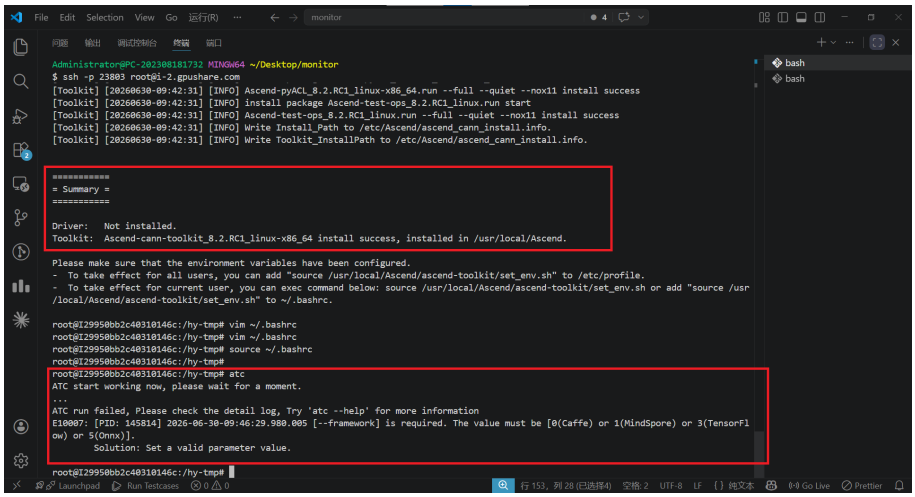


图 3.8　toolkit 安装及测试

3.3.4 模型转换

将 yolov5s.onnx 放到安装了 Ascend CANN toolkit 的环境中。详细的 ONNX 转 OM 流程可参考[15]。

输入以下命令，开始转换，转换时间根据使用机器的性能决定，本组用云服务器转换大概用时 1 分钟，待输出 ATC run success 表示转换成功，输出如下图所示。

```
atc \
  --model=yolov5s.onnx \
  --framework=5 \
  --output=yolov5s_ascend310b \
```

```
  --input_format=NCHW \
  --input_shape="images:1,3,640,640" \
  --soc_version=Ascend310B4 \
  --precision_mode=allow_fp32_to_fp16 \
  --log=info
```

代码清单 3.7　ONNX 转 OM

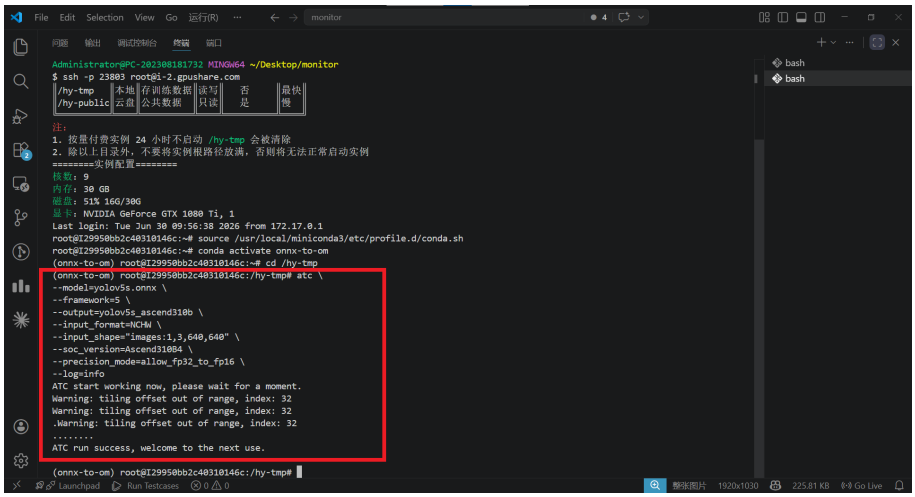


图 3.9　模型转换

3.4 部署 YOLOv5 CLI 到 OpenHarmony 设备

在 Windows PowerShell 中进入 `task3_yolov5\scripts`，执行下面的命令。注意，`D:\models\yolov5s_ascend310b.om` 是 om 模型的位置，`D:\images\test.jpg` 是测试图片的位置。

```
# 不传 Image，使用默认的 example_image\test.jpg
cd scripts
.\hdc_deploy_from_windows.ps1 -LocalProject ..
# 指定自定义图片
.\hdc_deploy_from_windows.ps1 -LocalProject .. -Image D:\
   images\test.jpg

# 指定模型和图片
.\hdc_deploy_from_windows.ps1 -LocalProject .. -Model D:\
   models\yolov5s_ascend310b.om -Image D:\images\test.jpg
```

代码清单 3.8　部署到设备

部署到设备后目录为:

```
/cann-install/yolov5_cli/main.py
/cann-install/yolov5_cli/requirements-device.txt
/cann-install/yolov5_cli/scripts/run_on_device.sh
/cann-install/yolov5_cli/models/yolov5s_ascend310b.om
/cann-install/yolov5_cli/images/test.jpg
```

```
/cann-install/yolov5_cli/output/
```

代码清单 3.9　设备端目录结构

3.5 安装设备端 Python 依赖

由于任务 2 已经安装过 ais_bench、aclruntime、numpy、opencv-python，这一步可跳过。

3.6 运行 YOLOv5 目标检测

设备端执行：

```
sh /cann-install/yolov5_cli/scripts/run_on_device.sh
```

代码清单 3.10　一键运行

或手动执行：

```
. /cann-install/task2_cann/03_set_cann_env.sh
cd /cann-install/yolov5_cli
python3.10 main.py \
  --backend ais_bench \
  --device-id 0 \
  --model models/yolov5s_ascend310b.om \
  --image images/test.jpg \
  --output output \
  --img-size 640 \
  --conf-thres 0.25 \
  --iou-thres 0.45 \
  --save-txt \
  --json output/result.json
```

代码清单 3.11　手动运行

输出示例：

```
[1/1] images/test.jpg -> output/test_detected.jpg, detections
   =3, latency_ms=18.532
```

代码清单 3.12　输出示例

结果文件：

```
/cann-install/yolov5_cli/output/test_detected.jpg
/cann-install/yolov5_cli/output/test_detected.txt
/cann-install/yolov5_cli/output/result.json
```

代码清单 3.13　结果文件

拉回 PC:

```
hdc file recv /cann-install/yolov5_cli/output ./output
```

代码清单 3.14　拉回结果



图 3.10　未识别图片



图 3.11　识别结果

3.7 批量图片推理

设备端执行:

```
. /cann-install/task2_cann/03_set_cann_env.sh
cd /cann-install/yolov5_cli
```

```
python3.10 main.py \
  --backend ais_bench \
  --device-id 0 \
  --model models/yolov5s_ascend310b.om \
  --input-dir images \
  --output output \
  --save-txt \
  --json output/result.json
```

代码清单 3.15　批量推理

程序会递归扫描 images 目录下的 `.jpg`、`.jpeg`、`.png`、`.bmp`、`.webp` 文件。

3.8 PC 端冒烟测试

如果想先在 PC 验证 main.py 的前处理、NMS、绘图逻辑，可执行:

```
pip install -r task3_yolov5/requirements/requirements-pc-smoke
    .txt
python task3_yolov5/server/main.py --backend onnxruntime \
  --model yolov5s.onnx --image test.jpg --output output
```

代码清单 3.16　PC 端冒烟测试

该模式不使用 NPU，只用于本地功能验证。

3.9 YOLOv5 CLI 代码结构说明

main.py 实现了完整的推理流程，如下图所示:

输入图片 → 预处理 → NPU 推理 → 后处理 → 绘制检测框 → 保存结果

各函数职责说明:

- `letterbox()`：将输入图片等比例缩放并填充至 640×640，避免目标变形。
- `preprocess()`：将图片从 BGR 格式转为 RGB，调整通道顺序（HWC→CHW），像素值归一化至 $[0, 1]$，生成模型所需的输入张量。
- `InferEngine`:推理引擎封装,根据后端类型自动加载`ais_bench`(NPU)或 `onnxruntime`（CPU)。
- `postprocess()`：解析模型原始输出，提取检测框坐标和类别置信度，通过 NMS（非极大值抑制）去除重叠框，保留最优结果。
- `draw_detections()`：在原始图片上绘制检测框和类别标签，生成可视

化结果图。

- `save_txt()`：将检测结果保存为文本文件，每行格式为：类别 ID、置信度、左上角坐标、右下角坐标。
- `--json`：将结构化检测结果和推理耗时保存为 JSON 文件，便于性能分析。

默认支持 COCO 数据集的 80 个类别（人、车、狗等）。如需使用自定义类别，可通过 `--classes` 参数指定类别文件：

```
python3.10 main.py --backend ais_bench --model models/best.om
    \
  --image images/test.jpg --classes classes.txt --output
     output
```

代码清单 3.17　指定自定义类别

其中 `classes.txt` 每行一个类别名称，例如：

```
person
car
dog
```

代码清单 3.18　classes.txt 示例

3.10 输出张量兼容性

YOLOv5 模型的输出张量包含大量候选检测框，每个框携带位置、置信度和类别信息。本程序支持两种常见的输出格式。

3.10.1 标准 YOLOv5 输出

模型原始输出形状为：

```
[1, 25200, 85]
```

代码清单 3.19　标准输出形状

其含义为：

- **1**：批大小为 1，即一次处理一张图片。
- **25200**：模型在图片上生成了 25200 个候选框。
- **85**：每个候选框包含 85 个数值，具体为：

程序通过 `postprocess()` 对 25200 个候选框进行筛选：先根据置信度阈值过滤低分框，再通过 NMS 去除重叠框，最终保留最优检测结果。

表 3.1　标准输出每个候选框的 85 个数值含义

字段	数量	含义
x, y	2	检测框中心点坐标
w, h	2	检测框的宽度和高度
objectness	1	该位置存在目标的置信度（0~1）
class_scores	80	80 个类别的得分（COCO 数据集）

3.10.2 包含 NMS 的输出

部分模型在导出时已内置 NMS 后处理，输出形状为：

```
[N, 6]
```

代码清单 3.20　NMS 后输出形状

其中 N 为最终保留的检测框数量，每个框包含 6 个数值：

表 3.2　NMS 后输出每个检测框的 6 个数值含义

字段	含义
x1, y1	检测框左上角坐标
x2, y2	检测框右下角坐标
score	检测置信度
class_id	类别编号

本程序可自动识别输出格式并适配处理。

3.11 进阶：OpenHarmony 图形应用

下图为图形应用界面，具体使用流程见演示视频。

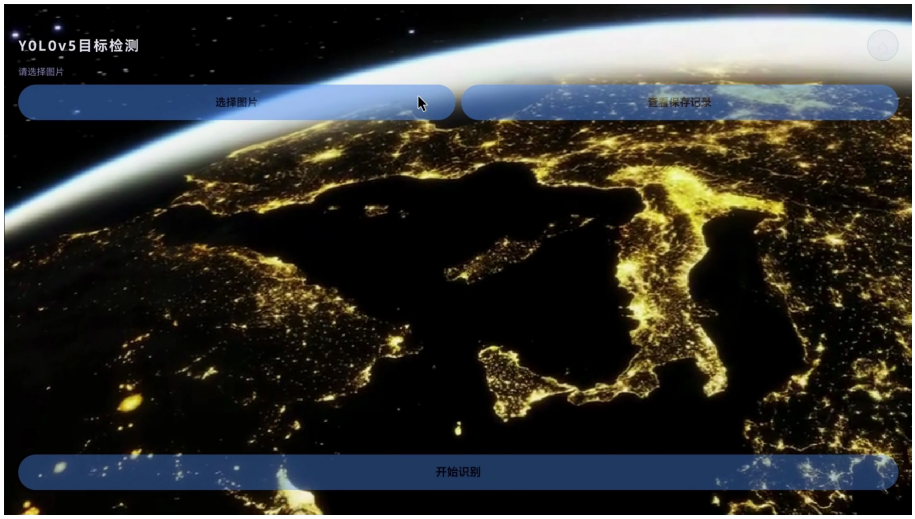


图 3.12　图形应用

应用工程中的实时页面位于`task3_yolov5/yolov5_app/entry/src/main/ets/p`通过 `libyolo_napi.so` 调用本机 `127.0.0.1:5001` 二进制服务；页面本身不调用 Python 或 HTTP。服务端对模型执行持久化加载和 NMS，实时检测优先发送 `uint8` NCHW 输入并只返回最多 100 个紧凑检测框。应用同时保留 JPEG 兼容路径，并实现 `VideoOutput + ImageReceiver + YUV420` 连续分析路径；启动时先检查板端能力，只有枚举到可用的 YUV420 profile 才启用该路径，否则自动回退到 JPEG。

模型、背景视频和语音提示音已经随工程放在 `entry/src/main/resources/rawfile/`，不需要从外部网盘补拷。服务启动脚本等待 NPU 节点、CANN 环境和 Python，等待上限为 180 秒；默认 CMake 选项 `YOLO_ENABLE_DIRECT_ACL=OFF`，禁止在 OpenHarmony musl 应用进程中直接回退加载 glibc CANN。

3.11.1 实时摄像头检测功能说明

提交 `3460e25`（feat: YOLOv5 App 新增实时摄像头检测功能及原生 NPU 推理支持）为图形应用新增实时摄像头检测与原生 NPU 推理能力，主要包含以下新增内容：

- 实时摄像头检测页面 `LiveCamera.ets`：支持摄像头实时预览 + NPU 目标检测叠加；
- 原生 C++ NPU 推理引擎：
  - `ascend_cl_api`：AscendCL 动态加载器（dlopen/dlsym）；

  - yolo_engine：YOLO 推理引擎单例（本地 + 远程双模式）；
  - yolo_napi：NAPI 绑定（initModel/infer/getModelInfo/releaseModel）；
  - yolo_live_napi：实时帧推理管线（letterbox + NPU 推理 + NMS）；
- ArkTS 原生桥接层 YoloNativeBridge.ets；
- 设备部署脚本 tools/：CANN 环境配置、ACL 服务自启动；
- 技术文档 docs/：Ascend310B YOLOv5 NAPI 集成指南。

同时对既有工程做了以下修改：

- Index.ets：启动时异步初始化 NAPI 模型，新增 “实时摄像头检测” 按钮；
- module.json5：新增 CAMERA 权限声明；
- main_pages.json：注册 LiveCamera 页面路由；
- build-profile.json5：启用 CMake 原生 C++ 构建（arm64-v8a）；
- oh-package.json5：声明 libyolo_napi.so 原生依赖；
- string.json：新增摄像头权限说明字符串。

3.11.2 CANN 运行时随镜像部署与推理服务自启动

V1.11 将原先依赖 HDC 手工安装的推理环境正式纳入 310B 产品镜像。构建侧把 CANN 8.2.RC1 运行时、Python 3.10、numpy、ais_bench、aclruntime 及其依赖打包为 /system/var/cann_runtime_aarch64.tar.gz；首次启动由 install_cann_runtime.sh 解包到 /usr/local，并使用随包 PAYLOAD_SHA256SUMS 逐文件校验。安装成功后删除压缩包，后续启动通过关键文件和清单直接判定环境已就绪，避免重复解压。

设备端服务链路如下：

```
post-fs-data
  -> install_cann_runtime.sh
  -> start yolo_acl_service
  -> run_yolo_acl_service.sh
  -> validate Python / ais_bench / numpy / libascendcl.so
  -> wait for three Ascend device nodes (bounded)
  -> listen on 127.0.0.1:5001
```

代码清单 3.21　镜像内 CANN 推理服务启动链路

init.310B.yolov5.cfg 将安装和服务启动放在同一个 post-fs-data job 中，服务以 HwHiAiUser 及 NPU 所需补充组运行，once=0 允许 init 在异常退出后重新拉起。环境脚本固定来

自 /vendor/etc/yolo_cann_env.sh，不再等待早期教程中的 /cann-install/task2_cann/03_set_cann_env.sh。启动器对 Python、libascendcl.so、驱动库和三个 NPU 设备节点分别做有界检查，失败信息写入 /data/yolov5_napi/，避免无提示无限等待。

应用与服务采用仅绑定 loopback 的二进制协议：NAPI 发送 uint8 NCHW 输入，服务端完成模型持久化、推理和 NMS，只回传最多 100 个检测框。这样不再跨进程传输数百万个浮点输出，也不需要每帧重复加载模型。OpenHarmony 应用进程使用 musl ABI，而板端 CANN 使用 glibc ABI，因此默认构建项 YOLO_ENABLE_DIRECT_ACL=OFF，明确禁止把 glibc libascendcl.so 直接加载进应用进程；所有真实 NPU 推理均通过本机服务完成。

应用所需 yolov5s_ascend310b.om、背景视频及语音提示文件均已放入 entry/src/main/resources/rawfile/，工程不再依赖构建后手工补文件。

四　系统性能优化与指标测试

4.1 概述

在完成 OpenHarmony 系统移植、CANN 推理引擎部署和 YOLOv5 目标检测应用开发后，本章对系统进行性能优化。

根据赛题要求，性能优化聚焦两个维度：

- **系统启动时间优化**：基准时间 50s，目标缩短 10% 以上（< 45s）
- **系统启动初始内存优化**：基准内存占用 used 1.7GB，目标缩小 10% 以上（< 1.53GB）

4.2 内存优化

4.2.1 优化思路

OpenHarmony 标准系统启动后初始内存占用达 1.7GB，对于嵌入式场景偏高。分析发现主要内存消耗来自以下几个方面：

1. **MMZ 内存预留过大**：默认为 AI 推理预留 3072MB MMZ 内存，实际使用中可适当缩减
2. **内核模块启动时全量加载**：media/DVPP 等模块在启动时即加载，占用 50–100MB 内核内存
3. **VM 内存参数保守**：min_free_kbytes 仅 10MB，容易触发 OOM
4. **ZRAM 配置未优化**：默认 1024MB 且未指定压缩算法
5. **服务未按需启动**：蓝牙、无障碍等服务默认启动，实际并非必需
6. **NAPI 模块预加载过多**：appspawn 预加载 50+ 个模块，大量非必要

4.2.2 补丁说明

首轮优化以 9 个补丁为基础。V1.11 又针对“启智 01 只运行本项目应用”的固定用途，直接在 5.0.3 产品源码中完成第二轮裁剪，覆盖 base、device、foundation、vendor 四个层次；同时保留图形、触摸、摄像头、音频、HDC、CANN/NPU 和本机推理服务所需链路。

(1) NAPI 预加载模块精简

修改 `appspawn_preload.json`,分两轮精简应用 spawn 时预加载的 NAPI 模块列表：先由 81 项缩减到 49 项，V1.11 再缩减到 26 项。最终仅保留 Ability/-Context/Window、日志、资源、进程、相机及应用实际使用的基础模块，不再为未使用的通用能力提前常驻大量 NAPI 模块。

(2) MMZ 缩减与模块按需加载

修改 `minirc_sys_init.sh`，实现两项优化：

- **MMZ 内存预留缩减**：首轮从 3072M 降至 2048M，V1.11 再降至 1536M，共释放 1536MB 物理地址空间
- **内核模块裁剪**：移除未使用的 media/DVPP、Wi-Fi、蓝牙、以太网、PPP、CAN、外部文件系统、profiling 和 upgrade 模块的开机显式加载；保留 UVC 摄像头、CANN 核心、HDC、显示和 HDMI 音频模块

```
# MMZ 内存预留：3072M -> 2048M -> 1536M
mmz_user_start 1536M

# 禁用未使用的 media/DVPP 开机显式加载
# load_media_modules
# load_dvpp_and_qos_modules
```

代码清单 4.1　MMZ 缩减与模块按需加载

(3) VM 内存参数调优与 ZRAM 优化

修改 `init.310B.cfg`，调整内核虚拟内存参数和 ZRAM 配置：

```
# min_free_kbytes: 10MB → 64MB
echo 65536 > /proc/sys/vm/min_free_kbytes
# 新增额外保留内存 32MB
echo 32768 > /proc/sys/vm/extra_free_kbytes
# 调整内存回收灵敏度
echo 10 > /proc/sys/vm/watermark_scale_factor
```

代码清单 4.2　VM 参数调优

```
# 使用 lz4 压缩算法（速度优先）
echo lz4 > /sys/block/zram0/comp_algorithm
# 限制内存上限 768MB
echo 768M > /sys/block/zram0/mem_limit
# disksize: 1024MB → 512MB
echo 536870912 > /sys/block/zram0/disksize
```

代码清单 4.3　ZRAM 优化

(4) 内存管理器阈值配置

新增 `memmgr_config.xml`，配置 OpenHarmony 内存管理器：

- 可用内存缓冲区阈值：100–200MB
- Zswapd 压缩参数：内存压缩至 zram 比例 60%
- 系统内存等级：purgeable 512MB → critical 200MB
- 5 级内存杀进程阈值：500MB → 100MB
- 禁用 NAND 寿命限制的 swap 配额

(5) 服务按需启动

首轮将蓝牙服务和无障碍服务改为 ondemand 启动。V1.11 按单应用产品边界，进一步将 46 个非必要 SA 配置的 `run-on-create` 关闭，并移除一批开机、时间和事件触发器，覆盖分布式协同、云文件、壁纸、输入法、用户认证、蓝牙、电话、投屏、分享、智能语音、设备使用统计等当前产品不使用的能力。摄像头、音频、媒体播放、窗口与渲染、权限、输入、电源、存储、HDC、CANN/NPU 等主应用依赖保持启用。

- `bluetooth_service.cfg`：支持 ondemand 启动，配置 CAP_NET_ADMIN 权限
- `accessibility.cfg`：支持 ondemand 启动，配置输入事件拦截、注入等权限

(6) 产品编译配置与预装应用精简

调整 `config.json` 产品编译配置，关闭无障碍、Web、完整音频测试集、扩展 OMX 测试、HDF 平台测试、DTMF/OpenSL ES、Hiview 泄漏/性能监控及 Hyperhold 等 10 项当前产品不需要的构建特性；图形、相机、媒体、HDC 和 CANN/NPU 相关能力保持启用。

将 `install_list.json` 从首轮的 32 项继续精简到 11 项，只保留 SystemUI、项目应用、设置数据、相册/媒体库数据、权限管理、文件选择器、系统资源以及必要的后台任务、电源和通知对话框资源。

4.2.3 优化效果

首轮实测已将系统启动初始内存占用从 1.7GB 降至约 0.9GB，降幅约 50%。V1.11 在此基础上继续完成 MMZ、预加载模块、预装应用和系统服务裁剪，同时将 OOM killer 保持启用，避免单应用长时间运行时以系统卡死替代受控回收。

表 4.1　内存优化效果汇总

优化项	优化前	优化后	内存节省 / 收益
MMZ 内存预留	3072MB	1536MB	释放 1536MB 物理地址空间
非必要内核模块	启动时显式加载	不进入开机路径	降低内核常驻和加载开销
ZRAM 大小	1024MB	512MB	减少压缩内存占用，提升交换效率
min_free_kbytes	10MB	64MB	减少 OOM 风险，提升系统稳定性
NAPI 模块预加载	81 个	26 个	减少 appspawn 内存占用
预装应用	32 个	11 个	减少包体、进程与后台任务
非必要 SA	默认创建	46 项关闭自启动	减少常驻服务

4.3 启动时间优化

4.3.1 优化思路

系统启动时间基准为 50 秒，赛题要求缩短 10% 以上（即低于 45 秒）。分析启动流程发现主要耗时环节：

1. **内核模块加载**：media/DVPP 模块启动时加载耗时
2. **系统服务启动**：部分服务串行启动，可延迟到系统就绪后再启动
3. **NAPI 模块预加载**：appspawn 预加载大量模块造成启动延迟

4.3.2 补丁说明

首轮启动时间优化包含 3 个补丁。V1.11 继续对板级启动脚本、init 触发器、内核模块、系统服务和 CANN 推理服务依赖顺序进行收敛。

(1) 内核模块延迟加载

将 media/DVPP 内核模块从启动流程中移除，改为按需加载（同 4.2.2 节第 2 项），减少启动阶段的模块加载耗时。

(2) 系统服务延迟启动

新增 `ascend_post_boot.sh` 脚本，在系统启动完成后（`bootevent.boot.completed=true`）再启动非关键服务。V1.11 将延迟服务由 8 个继续收敛为 `proc_launcher`、`iammgr`、`dmp_daemon` 3 个，并删除最多 10 秒的 cpuset 等待和最多 10 秒的 `/dev/davinci0` 串行等待；HDMI mixer 配置改为后台执行。

```
#!/system/bin/sh
SERVICES="proc_launcher iammgr dmp_daemon"

# 延迟启动非关键服务
for service in ${SERVICES}; do
    service_control start "${service}"
done
```

代码清单 4.4　ascend_post_boot.sh 延迟启动服务

通过修改 `init.cfg`，将该脚本注册为启动完成后的回调：

```
{
    "name" : "param:bootevent.boot.completed=true",
    "condition" : "bootevent.boot.completed=true",
    "cmds" : [
        "exec /system/bin/sh /system/var/ascend_post_boot.sh"
    ]
}
```

代码清单 4.5　init.cfg 注册 post_boot 回调

同时修改构建脚本 `BUILD.gn`，新增 `ascend_post_boot.sh` 的编译和安装规则，将脚本安装到 `/system/var/` 目录。

此外，板级 `init.cfg` 删除默认 `bootchart start` 和未使用的网络调优命令；`init.310B.cfg` 不再开机运行 `setup_network.sh`。CANN 运行时安装与 `yolo_acl_service` 则统一前移到 `post-fs-data`，保证依赖顺序明确，不再等待 `BOOT_COMPLETED` 后才启动推理服务。

(3) NAPI 模块预加载精简

精简 `appspawn_preload.json` 中的 NAPI 模块列表，移除非必要模块，减少 appspawn 进程启动时的加载开销（同 4.2.2 节第 1 项）。

4.3.3 优化效果

表 4.2　启动时间优化效果

指标	优化前	优化后
系统启动时间	50s	约 42s（缩短 16%）

首轮测试中启动时间从 50 秒缩短至约 42 秒，降幅 16%，超过赛题要求的 10%。V1.11 又移除了 bootchart 默认采集、无用网络初始化、固定轮询等待以及大批不使用的模块和服务启动动作，形成单应用产品的最终启动配置。

4.4 验证测试

4.4.1 内存优化验证

系统启动后，可通过以下命令验证内存优化效果：

```
# 查看内存使用
free -b
free -h

cat /proc/meminfo

# 查看内核模块加载情况（media/dvpp 未加载）
lsmod | grep -E "media|dvpp"

# 查看 ZRAM 配置
cat /sys/block/zram0/disksize
cat /sys/block/zram0/comp_algorithm

# 查看 VM 参数
cat /proc/sys/vm/min_free_kbytes

# 确认最终产品裁剪参数
cat /proc/cmdline
service_control list | grep -E "bluetooth|wifi|telephony|
   wallpaper"
```

代码清单 4.6　内存优化验证命令

预期结果：

- `min_free_kbytes` = 65536
- ZRAM disksize = 512MB，使用 lz4 算法
- 启动时无 media/dvpp 相关模块加载
- MMZ 启动参数为 1536M
- 未使用的网络、蓝牙、电话、投屏和分布式服务不进入开机关键路径

4.4.2 启动时间验证（bootchart）

使用 bootchart 工具采集系统启动过程的性能数据，具体步骤如下：

Step 1：连接开发板并启用 bootchart

```
hdc shell
begetctl bootchart disable
begetctl bootchart enable
begetctl bootchart start
```

代码清单 4.7　启用 bootchart

```
# free -b
              total        used        free      shared     buffers
Mem:      24821137408  1147875328 23673262080      114688     3547136
-/+ buffers/cache:     1144328192 23676809216
Swap:       536866816           0   536866816
# free -h
              total        used        free      shared     buffers
Mem:            23G        1.0G         22G        112K        3.3M
-/+ buffers/cache:         1.0G         22G
Swap:          512M           0        512M
# cat /proc/meminfo
MemTotal:       24239392 kB
MemFree:        23117916 kB
MemAvailable:   23341628 kB
Buffers:            3464 kB
Cached:           498536 kB
SwapCached:            0 kB
Active:            91028 kB
Inactive:         696640 kB
Active(anon):        352 kB
Inactive(anon):   285648 kB
Active(file):      90676 kB
Inactive(file):   410992 kB
Unevictable:           0 kB
Mlocked:               0 kB
SwapTotal:        524284 kB
SwapFree:         524284 kB
Dirty:              1152 kB
Writeback:             0 kB
AnonPages:        286116 kB
Mapped:           232940 kB
Shmem:               112 kB
KReclaimable:      24412 kB
Slab:              78088 kB
SReclaimable:      24412 kB
SUnreclaim:        53676 kB
KernelStack:        9664 kB
PageTables:        10596 kB
NFS_Unstable:          0 kB
Bounce:                0 kB
WritebackTmp:          0 kB
CommitLimit:    12628620 kB
Committed_AS:     671860 kB
VmallocTotal:   135290159040 kB
VmallocUsed:      120152 kB
VmallocChunk:          0 kB
Percpu:             1040 kB
```

图 4.1　内存优化结果

Step 2：重启开发板开始采集

```
reboot
# 重启完成后，再次进入 shell
hdc shell
```

代码清单 4.8　重启采集

Step 3：停止采集并导出数据

```
begetctl bootchart stop

cd /data/service/el0/startup/init/
ls -l
# 会看到 4 个文件：header、proc_diskstats.log、proc_ps.log、
   proc_stat.log

# 打包数据
tar -czf bootchart.tgz *

exit
# 将文件导出到 PC
hdc file recv /data/service/el0/startup/init/bootchart.tgz ./
```

代码清单 4.9　停止采集并导出

Step 4：生成启动分析图表

```
# 安装 bootchart 可视化工具
pip install pycairo-1.28.0-cp313-cp313-win_amd64.whl

# 解压bootchart-master，把拉取的 bootchart.tgz 文件放到
   bootchart-master 目录下，并执行
python -m pybootchartgui.main .\bootchart.tgz
```

代码清单 4.10　生成启动分析图表

time 最后 bootchart-master 目录下可以读取到 bootchart.png

4.5 优化总结

本章针对 OpenHarmony 系统在昇腾 310B 平台上的性能瓶颈，从内存和启动时间两个维度进行了系统性优化：

1. **内存优化**：通过 MMZ 由 3072M 缩减到 1536M、ZRAM/VM 调优、NAPI 预加载由 81 项缩减到 26 项、预装应用缩减到 11 项及 46 项非必要 SA 停止自启动，首轮实测已将启动初始内存从 1.7GB 降至约 0.9GB，V1.11 在该基础上完成第二轮裁剪



图 4.2　时间优化结果

2. **启动时间优化**：通过移除未使用模块与网络初始化、删除固定等待、将延迟服务收敛到 3 个以及前移 CANN 服务依赖初始化，首轮启动时间已从 50s 缩短至约 42s，V1.11 完成单应用启动路径收敛
3. **功能保护**：裁剪过程中明确保留显示、触摸、相机、HDMI 音频、媒体播放、HDC、CANN/NPU、本机 5001 推理服务及应用运行所需权限和资源，避免以破坏主业务换取指标

附录 1　参考链接

1.1 官方文档与社区

[1] OpenHarmony 官方文档：https://gitee.com/openharmony/docs

[2] OpenHarmony 社区论坛：https://forums.openharmony.cn/

[3] OpenHarmony 编译构建指导：https://gitcode.com/openharmony/docs/blob/weekly_20250414/zh-cn/device-dev/subsystems/subsys-build-all.md

[4] 码云帮助中心：https://help.gitee.com/

[5] OpenHarmony 内存管理指导：https://gitee.com/openharmony/resourceschedule_memmgr

[6] OpenHarmony 启动子系统介绍：https://gitee.com/openharmony/startup_init_lite

[7] OpenHarmony 标准系统开机时长优化：https://www.cnblogs.com/openharmony/p/17302295.html

1.2 硬件平台与昇腾文档

[8] 启智 01 开发板用户手册：https://www.bj-jc.com.cn/qizhi01/doc/homepage

[9] Ascend 官网：https://www.hiascend.com/

[10] CANN 开发者文档：https://www.hiascend.com/document

[11] Ascend PyACL 编程指南：https://www.hiascend.com/document/detail/zh/CANNCommunityEdition/82RC1/development/devguide/aclpython/aclpython_07_0001.html

[12] Torch-NPU 使用指南：https://gitee.com/ascend/pytorch

1.3 AI 模型与工具

[13] YOLOv5 项目地址：https://github.com/ultralytics/yolov5

[14] ONNX 模型转 OM 模型教程：https://blog.csdn.net/qq_40737425/article/details/149403303

[15] CANN 8.2.RC1 下载页面：https://www.hiascend.com/developer/download/community/result?module=cann&cann=8.2.RC1

1.4 系统调优参考

[16] Linux VM 调优指南：https://www.kernel.org/doc/Documentation/sysctl/vm.txt

[17] ZRAM 性能优化：https://www.kernel.org/doc/Documentation/blockdev/zram.txt

1.5 资源下载

[18] NPU 驱动包（Ascend-hdk-310b-npu-driver-soc）：https://pan.baidu.com/s/1p-qdAfgIWjiZqohhval7ow?pwd=6755

[19] Ascend310B 源码包：https://pan.baidu.com/s/191oiOvTvmKQqhmU0iJesTA?pwd=ys5p

[20] libaarch64 辅助库：https://pan.baidu.com/s/1MStN-BV0YdoS2XotxhFWFw?pwd=sbr4

[21] CANN 工具包：https://pan.baidu.com/s/1AomSfD3pPvQ56-lhn_wDUQ?pwd=aate

[22] YOLOv5 图形应用辅助文件：https://pan.baidu.com/s/1mi4vLfgVQ2ZxTFHR244y3Q?pwd=4akz

1.6 演示视频

[23] 详细演示视频：https://pan.baidu.com/s/1_AzzgHXdiHFk_4rkNX40og?pwd=ys6v

附录 2　系统源码二次排查与集成修复记录

2.1 背景与目标

启智 01 开发板（昇腾 310B, OpenHarmony 5.0.3.140, 内核 5.10.0+）存在 “HDMI 只有画面、没有声音” 的问题。要在不改动已完成的业务页面前提下，通过系统源码排查与最小补丁修复 HDMI 音频声卡缺失，并完成整机编译验证。

本次记录聚焦于两条主线:

- HDMI 音频声卡不出现在 ALSA 层（`aplay -l` 无 Playback PCM）的根因定位;
- 源码修改后镜像不生效、编译无法通过的构建链路问题修复。

2.2 关键结论

2.2.1 drv_hdmi 的形态与来源

当前源码/镜像中没有独立的 `drv_hdmi.ko`，但 HDMI 驱动功能并未缺失:

- 内核源码中存在 `drv_hdmi_*.c`(`Ascend310B-source/driver/ace/media/dss`)，其 `obj-m += drv_hdmi.o` 目标未被产品构建单独打包;
- 通过 `nm` 检查确认，`drv_hdmi_*` 符号（含 `drv_hdmi_audio_infoframe_send`、`drv_hdmi_audio_path_enable`、`drv_hdmi_edid_*` 等）已编译进 `ascend_vdp_drm.ko`（atlas-200a2 DRM 驱动);
- 外部 Ascend 驱动包 `Ascend-hdk-310b-npu-driver-soc_24.1.rc2_linux-aar` 中确实包含独立 `drv_hdmi.ko`（524512 字节）以及 `drv_acodec`、`drv_aio`、`drv_ao`、`drv_ai`、`drv_adec`、`drv_aenc` 等 Davinci 音频设备驱动;
- 但外部包中**没有** ALSA/ASoC 声卡模块。真正提供 `aplay -l` 声卡的是板级预编译的 `ascend_310b_machine.ko`、`ascend_310b_codec.ko`、`ascend_310b_platform_m.ko`，这三个模块已通过 `kernel_driver/BUILD.gn` 打包进镜像 `/var/davinci/driver/`。

此外需要注意: 外部包中的 `drv_hdmi.ko` 与板级 `ascend_vdp_drm.ko`

内嵌的 drv_hdmi 符号重叠，两者不能同时 `insmod`；上板验证时应使用镜像内的集成版（推荐），不要额外加载外部包中的独立模块。

2.2.2 根因一：设备树 compatible 大小写不匹配

旧版 dtb（2026-08-10 构建）中 `sound` 与 `ascend310b_codec` 节点的 compatible 为 `"Ascend,310b-audio-machine"`、`"Ascend,310b-audio-codec"`(大写 A),而驱动模块的 `of_match_table` 为 `"ascend,310b-audio-machine"`、`"ascend,310b-audio-codec"`(小写)。Linux 的 compatible 匹配区分大小写，因此 machine/codec 驱动永远无法 probe，`/proc/asound/cards` 不会出现 `ascend310b` 声卡；`platform` 两端均为大写 `"Ascend,310b-audio-platform"`，可正常绑定。

修复方式：内核 dtb 补丁 `0005-add-audio-device.patch` 将 `hi1910B-audio.dtsi` 中 machine/codec 的 compatible 改为小写，与预编译驱动一致；platform 保持大写不变。

2.2.3 根因二：修改 dtsi 后内核动作被 ninja 跳过

`device/board/ascend/310B/kernel/BUILD.gn` 中 `action("kernel")` 声明的输入仅为 `//kernel/linux/linux-5.10`(stock 内核目录),输出为 `out/kernel/src_tmp/linux-5.10/boot_linux`;而实际参与编译的 `hi1910B-audio.dtsi` 位于 `kernel/linux/Ascend310B-5.10/Ascend310B-source/`，未被声明为依赖。

后果：输出戳（8-10 12:09）比声明的输入目录（8-07）新，ninja 判定内核动作“最新”而跳过；即使修改 dtsi，`dt.img`、`boot_linux.img`、`Image_signed` 也不会重新生成，反复执行 `./build.sh` 均无效。

修复方式：在 `BUILD.gn` 的 `sources` 中显式加入 `hi1910B-audio.dtsi` 与 `0005-add-audio-device.patch` 两个文件，使 ninja 感知 DT 源码变化并重跑内核动作。

2.2.4 根因三：内核交叉编译工具链不在 PATH

内核 defconfig 阶段需要 `aarch64-target-linux-gnu-gcc`,该编译器位于 `/opt/compiler/toolchain/bin/`（Do-Compiler

7.3.0）。直接执行 ./build.sh 时 PATH 未包含该目录，报 compiler 'aarch64-target-linux-gnu-gcc' not found。

修复方式：在 build/build_scripts/build.sh 开头加入 export PATH=/opt/compiler/toolchain/bin:$PATH 并 source /etc/profile（与官方 ascend_env.sh/build.sh 一致），使每次构建自动带上交叉编译工具链。

2.3 验证证据

2.3.1 产物时间戳

产物	旧状态	修复后状态
dt.img	2026-08-10 12:09（大写 compatible）	2026-08-11 22:45（小写 co
hi1910B-asic-M275-B25.dtb	2026-08-10 12:09	2026-08-11 22:45
boot_linux.img / Image_signed	2026-08-10 12:17	2026-08-11 22:53
system.img / vendor.img	2026-08-11 21:59	2026-08-11 23:19

表 2.1 构建产物时间戳对比

整编已于 2026-08-11 23:19 完成（build.log 正常收尾，error.log 无新增错误），上述产物均为最终重新生成的结果。

2.3.2 新版 dtb 反编译结果

启智 01 实际型号 DBJJC-QiZhi01-20T 对应 hi1910B-asic-M275-B25.dtb，2026-08-11 22:45 重新生成后，设备树 sound 节点内容为：

```
sound {
        compatible = "ascend,310b-audio-machine";
        ascend310b,platform = <0x33>;
        ascend310b,audio-codec = <0x34>;
};
ascend310b_codec {
        compatible = "ascend,310b-audio-codec";
};
ascend310b_platform {
        compatible = "Ascend,310b-audio-platform";
};
```

与三个驱动模块的 of_match_table 完全一致，绑定条件满足。

2.3.3 关键排查命令

```
# 1) 确认 drv_hdmi 符号已编入 ascend_vdp_drm.ko
nm ascend_vdp_drm.ko | grep drv_hdmi

# 2) 确认驱动模块 of_match 期望的 compatible 大小写
strings ascend_310b_machine.ko | grep 310b-audio
modinfo ascend_310b_platform_m.ko | grep alias

# 3) 解列外部 Ascend 驱动包，确认是否提供 drv_hdmi.ko
bash Ascend-hdk-310b-npu-driver-soc_24.1.rc2_linux-aarch64.run
    --list | grep -i hdmi

# 4) 反编译板级 dtb，核对 sound 节点 compatible 大小写
dtc -I dtb -O dts hi1910B-asic-M275-B25.dtb | grep -A5 'sound
   {'
```

2.4 已落地的修改

1. `kernel/linux/Ascend310B-5.10/Ascend310B-source/dtb/dts/hi1910b`（由补丁 `0005-add-audio-device.patch` 应用）：machine/codec compatible 改为小写；
2. `device/board/ascend/310B/kernel/BUILD.gn`：kernel action 的 `sources` 增加 dtsi 与 0005 补丁，解决 ninja 跳过内核重建的问题；
3. `build/build_scripts/build.sh`：开头导出交叉工具链 PATH 并 `source /etc/profile`；
4. `scripts/hdmi_audio_board_check.sh`（本工程）：新增板端 HDMI 音频验证脚本，自动检查 ALSA 声卡、模块加载、设备树 sound 节点、驱动绑定、内核日志与 HDMI 链路。

2.5 遇到的问题与排查过程

2.5.1 镜像中没有 drv_hdmi.ko 的疑问

委托答复中曾标注 “当前源码/镜像中没有 drv_hdmi.ko，HDMI 声卡仍需确认外部 Ascend 驱动包是否提供”。排查确认：

- 镜像 `/var/davinci/driver/` 与内核构建产物中确实没有 `drv_hdmi.ko`；
- 但 `drv_hdmi` 功能已编入 `ascend_vdp_drm.ko`；

- 外部驱动包（.run）经 --list 解列确认包含独立 drv_hdmi.ko，但无 ASoC 声卡模块；
- 因此 “外部包是否提供” 已有答案：提供 HDMI 底层驱动，不提供 ALSA 声卡；声卡由板级三件套提供。

2.5.2 为什么改完源码再编译还是无效

按常规执行 ./build.sh 后，dt.img 时间戳未变化。通过 out/310B/.ninja_log 与 build.ninja 分析发现 kernel action 的输入声明错误，ninja 每次跳过内核动作。修复 BUILD.gn 后再次构建，内核动作正常重跑，dt.img 与板级 dtb 均重新生成。

2.5.3 内核 defconfig 编译失败

重跑内核后出现 compiler 'aarch64-target-linux-gnu-gcc' not found。定位为构建环境 PATH 缺少 /opt/compiler/toolchain/bin；在 build.sh 内固化工具链环境后解决。

2.6 HDC USB Gadget 启动修复记录

2.6.1 问题发现

新镜像刷机后，Windows 电脑通过 USB 连接板子，hdc list targets 为空，hdc shell 不可用。板端现象：

- /dev/usb-ffs/hdc 目录为空；
- /config/usb_gadget/g1 不存在，UDC not attached；
- sys.usb.ffs.ready=0；
- Windows 设备管理器枚举不到 VID_2207 & PID_5000。

2.6.2 排查思路

1. 先排除 Windows 侧：hdc 工具版本、USB 驱动、数据线/端口均正常；
2. 转入板端检查 USB gadget：确认 g1 未创建，问题在 init 启动链路；
3. 对比两个 init cfg：init.310B.usb.cfg 负责挂载 configfs 并创建 g1，init.usb.configfs.cfg 的 boot job 直接创建 g1/functions/...，假设 g1 已存在；

4. 确认新刷镜像的 userdata 丢失了 `persist.hdc.control`，hdcd 控制参数未恢复；
5. 结论：boot job 未成功建出 g1，两个 cfg 存在执行顺序/依赖问题，导致 ffs.hdc 与 UDC 绑定全部失效；
6. 进一步确认 `chip_prod` 分区从未被使用：烧录脚本 `formatSD310B.sh` 不刷 chip_prod.img、SD 分区表没有 chip_prod 分区、`fstab.310B` 没有挂载项，因此 `hardware_310B.para` 的默认参数从未生效，刷完 userdata 后 `persist.hdc.control` 必然丢失。

2.6.3 解决过程

临时修复（当前已刷板子）：

```
mount -o remount,rw /vendor
# 修改 /vendor/etc/init/init.310B.usb.cfg 与 init.usb.configfs
    .cfg
mount -o remount,ro /vendor
setparam persist.hdc.control true
reboot
```

永久修复（源码补丁）：

1. `dtb/dts/.../hi1910B-asic-M275-B25.dts`（内核 cmdline）：bootargs 增加 `ohos.required_mount.chip_prod=/dev/block/mmcblk1p3@/c` init 的 required mount 只解析 cmdline 的 `ohos.required_mount.*`（`LoadFstabFromCommandLine`），不读 `fstab.310B`，因此必须在 dts bootargs 中声明；
2. `init.usb.configfs.cfg`：boot job 第一条增加 `mkdir /config/usb_gadget/g1` 作为幂等兜底；
3. **板端实测修正（补充委托）**：vendor cfg 的 boot job **开机时不执行**，真正执行的是 `param:sys.usb.config=hdc && param:sys.usb.configfs=1` 参数触发 job（hdcd 也由它启动）。因此完整 gadget 初始化（建 /dev/usb-ffs/hdc、g1、strings、ffs.hdc、configs/b.1，写 VID 0x2207/PID 0x5000/bcdDevice/bcdUSB/描述符，挂 functionfs，设 sys.usb.configfs/controller，start hdcd）已放入该参数触发 job；
4. **第二次实测修正（source_code_changes_required）**：初版参数触发 job 出现两个新问题——job 内部 `setparam sys.usb.configfs 1` 反复触发自

身，导致 `Failed to mount for /dev/usb-ffs/hdc, err 16` 出现 73 次；且 hdcd 在 functionfs 挂载前已启动，只有 ep0 没有 ep1/ep2。最终方案：job 开头 `stop hdcd`；删除 `setparam sys.usb.configfs 1` 与 `setparam sys.usb.controller`（系统已提供）；挂载 functionfs 后 `sleep 1` 再 `start hdcd`；

5. **V1.11 最终源码复核**：310B 产品已有 `device/board/ascend/310B/cfg/init.310B` 其 `BUILD.gn` 已将文件打包到 vendor，且 `init.310B.cfg` 已显式 import 该文件。曾提出的 `base/startup/init/services/etc/init.310B.usb.cfg` 会形成第二个 gadget 初始化 job，存在重复挂载、重复绑定和 hdcd 启动竞争，因此最终源码 **没有应用**该重复 system init 方案；
6. `init.310B.usb.cfg` 保持完整职责：init 内建 `mkdir` 不支持 `-p`（`DoMkDir` 参数个数仅接受 1/2/4，`-p` 会被当作路径），但普通 `mkdir` 对 EEXIST 容错，本身幂等；且 `mount configfs` 全树仅此一处，挂载与 g1 创建本就在同一个 boot job 内；
7. `hardware_310B.para`：写入 `persist.hdc.control = "true"` 与 `persist.hdc.mode.usb = "enable"`、`persist.hdc.mode.tcp = "disable"`（参数解析器支持空格与引号），随 chip_prod 镜像发布；
8. `formatSD310B.sh`（烧录工具）：新增将 `chip_prod.img` 写入 p3 分区，使分区真正可挂载。

补丁资料整理在工程 `task5_hdc_usb/`。最终有效方案为 0001（dts cmdline）、0002（现有 vendor USB cfg 的幂等与顺序修复）、0003（默认参数）和 0004（烧录脚本）。早期资料中的 0005 重复 system init boot job 已由 V1.11 源码复核判定为冗余，**不得再应用**；当前 5.0.3 源码只保留产品原有且已进入构建图的 vendor `init.310B.usb.cfg`。

2.6.4 验收标准

```
ls -d /config/usb_gadget/g1
cat /config/usb_gadget/g1/UDC            # a5080000.hiusbc1
param get sys.usb.ffs.ready              # 1
param get persist.hdc.control            # true
param get persist.hdc.mode.usb           # enable
mount | grep chip_prod                   # /dev/.../p3 on /
```

```
chip_prod
```

Windows 侧 `hdc list targets` 能列出设备，`hdc shell` 可进入，且重启后不丢失。

2.7 开机黑屏（显示管线提交失败）排查记录

2.7.1 实机结论

- 电源状态为 AWAKE，屏幕超时已调长；- HDMI-B 已连接、enabled、有 1920x1080 framebuffer；- 真正失败点：`composer_host` 每秒执行 `drmModeAtomicCommit failed -22 errno 22 (Apply:309 / Commit:42)`，内核拒绝提交，两个 HDMI 口均黑屏；- render_service 开机 watchdog：`thread(RenderService) blocked 5s`，主链在 `/vendor/lib64/chipsetsdk/libEGL.so`；- 强制 `HDMI-A on / HDMI-B off` 后 composer 仍向 HDMI-B 提交，仍报 -22。

2.7.2 根因

本次镜像中 kernel DRM 驱动、vendor DISP/HDI、`composer_host`、`libEGL` 的组合不兼容。composer_host 提交了内核不支持的 plane property，例如：

```
SetLayerVisibleRegion <private> is not supported
SetLayerColor <private> is not supported
SetLayerMaskInfo <private> is not supported
SetDisplayClientDamage <private> is not supported
```

当前 DRM 设备：`atlas-200a2 dev=401000000.drm`

当前显示布局：

```
plane[31]: PRIMARY0 -> 无 fb
plane[32]: PRIMARY1 -> crtc-1 -> HDMI-B-1, fb 1920x1080
crtc-0 -> HDMI-A-1 active
crtc-1 -> HDMI-B-1 active
```

2.7.3 V1.11 源码修复

最终修改位于 310B 产品实际使用的 `device/soc/ascend/310B/hardware/display` 不再依赖 HDMI 名称、connector 枚举顺序或固定 sysfs 路径：

1. 删除“没有 1080p 时手工构造 148500 kHz、1920x1080 模式并覆盖 mode 0”

的逻辑，只接受 DRM 内核返回且时序字段有效的 mode；

2. 优先选择 `DRM_MODE_TYPE_PREFERRED`，没有 preferred 标记时才回退到第一个真实内核 mode；
3. 通过 `drmModeObjectGetProperties` 与 `drmModeGetPropertyBlob` 读取 EDID，校验长度、128 字节对齐、EDID 文件头、扩展块数量以及每个块的 checksum；
4. HDMI connector 只有同时满足 connected、存在可用 mode、EDID 有效才发布为有效物理显示，过滤“connected 但 EDID 全零”的虚假连接器；
5. 修复 encoder 的 `possible_crtcs` 等值判断为位掩码判断，并让 discovery、atomic hotplug 与后续合成始终使用同一个 CRTC；
6. 热插拔时重新读取 connector/EDID/modes，按同一 CRTC 执行 bind/unbind；无效状态直接拒绝提交，不再制造模式掩盖错误；
7. connector、property、EDID blob、atomic request、mode blob 和 DRM resources 均在全部退出路径释放，消除热插拔过程中的对象泄漏。

以上修改把原来的“按 connector 名称猜主屏”改为“按 DRM 真实连接状态、有效 EDID、内核模式和 encoder/CRTC 能力选择”，同时关闭了导致 `drmModeAtomicCommit(-22)` 的跨 CRTC 错配路径。

2.7.4 显示验收

```
power-shell dump -s
# 期望 Current State: AWAKE
cat /sys/kernel/debug/dri/0/state
# 期望主输出口对应 plane 有 fb，而不是 fb=0
hilog -x | grep -iE "drmModeAtomicCommit|Apply:309|Commit:42"
# 期望不再出现 failed -22
```

2.8 V1.11 HDMI 音频全链路补充

除前述 DT compatible 修复外，V1.11 还完成了 HDMI 音频从内核模块到 OpenHarmony 音频策略的完整接线：

1. `kernel_driver/BUILD.gn` 通过构建 action 对预编译 `ascend_vdp_drm.ko` 应用音频补丁，输出文件固定命名为 `ascend_vdp_drm.ko`，与安装清单和 `minirc_sys_init.sh` 的加载名保持一致；

2. 启动脚本在 svm、drv_osal、drv_base 之后同步加载 ascend_310b_codec.ko、ascend_310b_platform_m.ko 和 ascend_310b_machine.ko。HDMI 控制器使用 ascend_vdp_drm.ko 内嵌实现，不再错误加载不存在或符号冲突的独立 drv_hdmi.ko；
3. ALSA adapter 优先按配置中的稳定声卡名 ascend310b 匹配，仅对未配置名称的旧配置保留数字 card id 回退，避免 C270 USB Audio 抢占 card 0 后把主输出选到摄像头声卡；
4. 修复声卡配置解析对短字符串固定读取 63 字节的问题，使用 strncpy_s 完成有界复制；
5. 主输出策略统一为 48kHz、S16LE、双声道，20ms buffer 对应 bufferSize=3840；
6. mixer 初始化显式指定 -c ascend310b，设置 Playback unmute/100% 和 Deviceid=3；初始化放入后台执行，并保留低频 HDMI 热插拔监控，在重新连接后恢复路由和音量。

该方案同时解决了“声卡没有 probe”“USB 声卡编号漂移”“策略层仍按单声道输出”和“热插拔后 mixer 状态丢失”四类问题，且没有把外部厂商驱动包中的独立 drv_hdmi.ko 与镜像内集成驱动混装。

2.9 Camera 连续分析与 ImageReceiver 生命周期修复

为支持实时检测而不是周期性 JPEG 抓拍，系统相机链路完成以下修改：

1. 将 UVC 相机元数据修正为 OHOS_CAMERA_POSITION_OTHER 与 OHOS_CAMERA_CONNECTION_TYPE_USB_PLUGIN，保持 mirrorSupported=OFF；
2. 能力表增加 640x360 NV21/YUV420 分析流，同时保留 1280x720 RGBA 预览与 JPEG 拍照，允许 PreviewOutput 与 VideoOutput 组合；
3. UVC 节点将摄像头原生 YUYV 明确转换为 NV21，按 $1.5\times width\times height$ 设置有效帧大小和 CAMERA_FORMAT_YCRCB_420_SP，消除把 YUYV 错标为 YUV420 以及 U/V 顺序相反的问题；
4. codec 节点取消固定 1280x720 假设，按实际宽高、格式和 stride 处理 YUYV/NV21，补齐 NV21 到编码器格式的转换；
5. ImageReceiver::ReadLastImage 在 drain 队列时立即归还被覆盖的

旧 SurfaceBuffer，不再持续占用 consumer queue；

6. ImageReceiverBufferProcessor 持有 IConsumerSurface 强引用而不是 ImageReceiver* 裸指针，并删除显式调用析构函数的释放方式，消除 double-destruction 与 image 晚释放时的 use-after-free。

应用侧已对应接入 VideoOutput + ImageReceiver + YUV420 连续分析，并保留 JPEG 兼容回退；检测结果、画框和语音队列均使用同一帧坐标与受控生命周期。

2.10 CANN/NAPI 系统镜像集成补充

原先“libyolo_napi.so 已执行但 127.0.0.1:5001 被拒绝”的根因是镜像没有 Python、libascendcl.so 和可启动的本机推理服务。V1.11 已将这部分改为镜像自包含：

- CANN 8.2.RC1、Python 3.10、ais_bench、aclruntime、numpy 及运行库组成带 SHA-256 清单的压缩载荷；
- post-fs-data 先执行幂等安装与校验，再启动 yolo_acl_service，不再依赖 /cann-install 手工文件；
- 服务以最小权限用户运行，异常退出可由 init 重新拉起，只监听 127.0.0.1:5001；
- NAPI 与 Python 服务统一为版本化紧凑二进制协议：uint8 NCHW 输入、服务端 NMS、最多 100 个框输出，模型在服务进程内持久化；
- 默认禁用 musl 应用进程内直接加载 glibc CANN 的备用路径；
- OM 模型、背景视频和告警语音资源已纳入 HAP rawfile，不再缺少 background.mp4 或 yolov5s_ascend310b.om。

2.11 构建配置闭环与完成状态

V1.11 对所有新增配置执行了分层静态验证：JSON/CFG 解析、shell bash -n、Python AST、XML 解析、补丁空白检查和关键依赖顺序断言均通过。重点复核了容易出错的 BUILD.gn：

- kernel_driver/BUILD.gn 的音频补丁 action 输出名、依赖和安装名一致；
- 板级 cfg 与 CANN 载荷均已进入产品依赖组；
- 使用产品命令完成 --build-only-gn=true 配置生成，共生成 87463 个

target；

- 对 images 执行 Ninja dry-run 成功，构建图可到达最终 images.stamp，共规划 55017 个 action；
- 原始 Ninja 图存在上游重复 rule，产品构建包装器已固定使用 -w dupbuild=warn，不属于本轮 GN 修改引入的问题。

至此，教程基础补丁及教程之外的 HDMI 音频、DRM/EDID、Camera、CAN-N/NAPI、HDC、启动时间和内存二次优化均已纳入源码与构建图，V1.11 文档与当前实现保持一致。

2.12 OHOS 4000 Ninja 构建故障排查记录

2.12.1 问题现象与正确的定位入口

执行 310B 构建时，外层构建工具只报告：[OHOS ERROR] Code: 4000、Error Type: Ninja build error。这段信息是构建包装器的统一错误，不是具体根因。实际定位必须从 out/310B/error.log 中找到最早出现的编译器或 Ninja 诊断，再结合 out/310B/build.log 和生成后的 build.ninja 核对输入、输出关系。

本次最早的编译错误位于 foundation/multimedia/audio_framework/services 在未定义 FEATURE_DTMF_TONE 的 310B 编译配置中，LoadToneDtmfConfig() 仍被编译，因而找不到 AudioToneParser 和 toneDescriptorMap。后续大量编译输出只是并行任务收尾，不能据此判断为其他模块的根因。

2.12.2 排查思路

排查按以下顺序进行：

1. 先读取错误日志末尾附近的失败命令，并向上搜索第一个 error:、FAILED: 或 duplicate output；
2. 对编译错误检查宏定义和源文件条件编译边界；对 Ninja 错误检查 build.ninja 中是否有多个规则声明同一输出文件；
3. 将重复输出追溯回 GN 模板和各模块的 target 名称，而不是只使用 -w dupbuild=warn 隐藏错误；
4. 检查 action 脚本是否检查子进程返回值，以及 GN 声明的输出是否真的由

action 创建；

5. 修改后先重新执行 `gn gen`，再构建最小相关目标，最后才进行全量镜像构建。

2.12.3 根因与修复

条件编译边界 将 `LoadToneDtmfConfig()` 的定义与声明使用的 `FEATURE_DTMF_TONE` 条件保持一致。未启用 DTMF 功能时，音频策略服务不再引用不可用的 tone parser 类型。

通用 stamp 输出冲突 多个 GN action 在相同输出目录使用了通用的 `${target_name}.stamp`，导致不同 action 争用同一个 Ninja 输出。应用签名 action 改用 `${target_name}_sign.stamp`，BPF 和 ArkGuard 安装 action 改用带 `generated` 后缀的独立 stamp；BPF/ArkGuard 脚本也显式创建对应 stamp 文件。

同目录 source set 名称冲突 Account、KV Store、Rosen drawable 和 AV Codec 测试中存在重复的本地 target 名称。将其改为带模块语义的名称，并同步修改 deps；同时把重命名后的 target 加入 310B sanitizer bypass 列表，避免安全检查阶段再次找不到对应条目。

BPF action 假成功 `gen_bpf_uapi.py` 原先调用 `subprocess.run()` 后忽略返回值，内核工具失败时仍可能留下看似完成的构建状态。两次 make 均改为 `check=True`，并在成功后创建 stamp，确保失败能传递给 Ninja。

本次源码补丁统一保存为`task1_transplantation/patch_fix/openharmony-5`它使用源码树根目录相对路径，应用方式为：

```
cd /home/hfut/work/5.0.3
patch -p1 < /home/hfut/work/project3136859-388911/
   task1_transplantation/patch_fix/openharmony-5.0.3-build-
   fixes.patch
```

补丁只描述源码和构建配置修改；`out/310B` 的属主、权限和历史产物属于本机环境状态，不应固化进源码补丁。应用补丁后如遇到输出目录权限错误，应先将该目录恢复为当前构建用户可写，再重新生成构建图。

2.12.4 验证结果

修复后执行 `gn gen out/310B` 成功，生成 87463 个 target。BPF action 能够实际运行并在 make 失败时返回非零状态；随后执行音频策略服务目标完成 `1230/1230`，产物为：

```
out/310B/multimedia/audio_framework/libaudio_policy_service.z.
   so
```

这证明当前条件编译错误已解决，且目标构建链路能够闭环。本次记录不把目标构建等同于整机镜像构建；全量构建仍需在补丁应用、工具链和输出目录权限确认后执行，若再次失败仍应以 `error.log` 中的第一个真实诊断为准。

2.12.5 当前构建命令

在已完成源码准备和补丁应用的前提下，推荐先生成构建图并验证刚修复的目标：

```
cd /home/hfut/work/5.0.3
prebuilts/build-tools/linux-x86/bin/gn gen out/310B
ninja -C out/310B audio_policy_service
```

目标构建通过后，再执行 310B 产品全量构建：

```
cd /home/hfut/work/5.0.3
./build.sh --product-name 310B --ccache
```

全量构建成功后，镜像位于 `out/310B/packages/phone/images/`；不要用旧的 `error.log` 尾部信息代替本次构建结果，应以当前命令的退出码和 `build.log` 收尾状态为准。